Practical Assessment OOP

Name: Chime Salma Chiamaka

Module: Object Oriented Programming

Repository link: https://github.com/salm-aA/shapes-project

Object-Oriented Concepts Used

In this project I used the main object-oriented programming principles: encapsulation, abstraction, inheritance and polymorphism.

- Encapsulation is used by keeping class attributes private and accessing them through methods.
- Abstraction is shown through the abstract Shape class.
- Inheritance is used because the different shapes extend the Shape class.
- Polymorphism is used because different shape objects can be stored and managed using the Shape type.

Program Design

This program is designed to manage a list of geometrical shapes and carry out calculations such as area, perimeter, translation and scaling. I split the program into separate classes so that each class has its own clear responsibility.

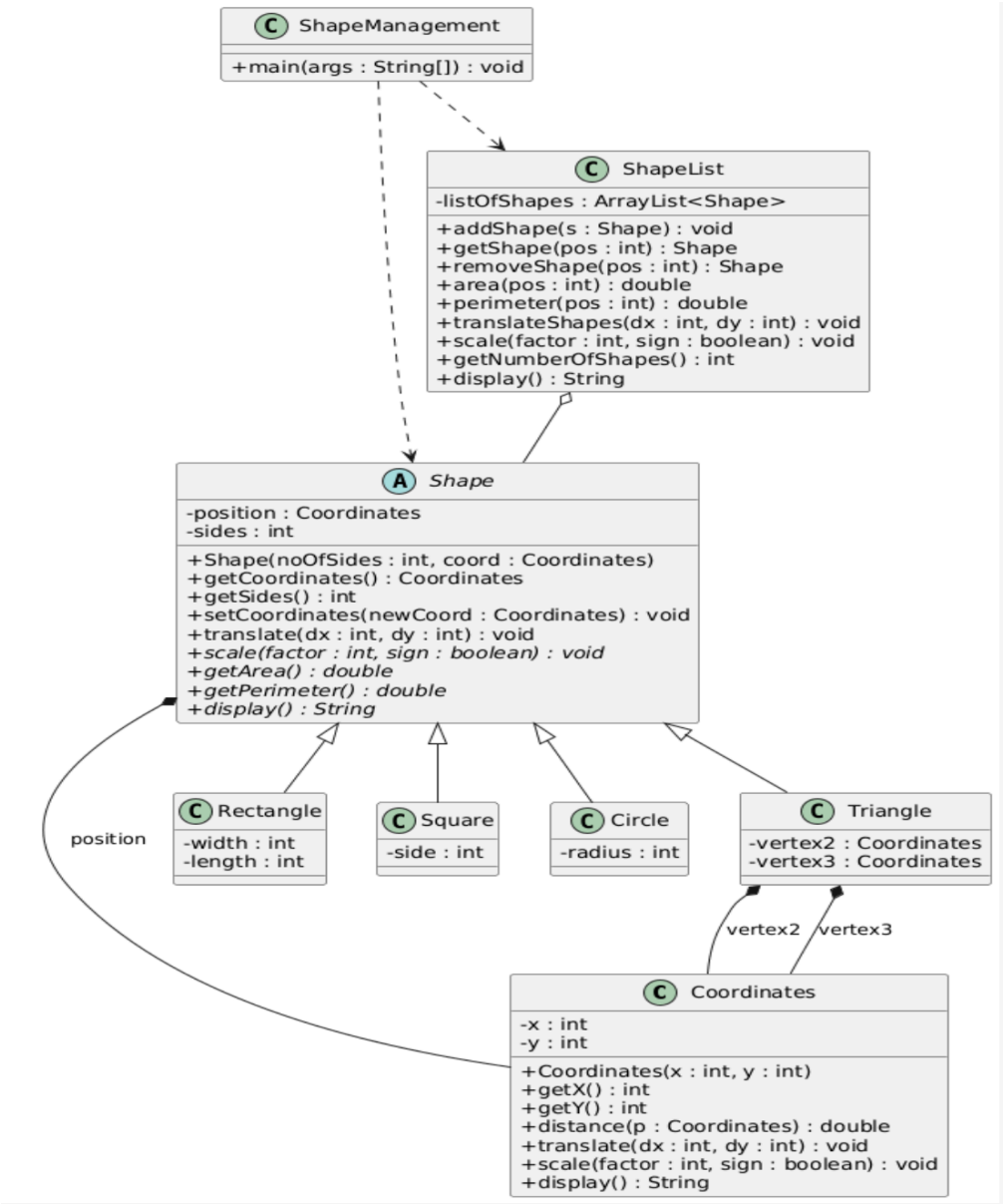
The Coordinates class represents a point on the screen using x and y values. It includes methods for calculating the distance between two points, moving a point, scaling a point and displaying the coordinate values.

The Shape class is an abstract superclass because a general shape does not have a specific area or perimeter by itself. It stores the common information that all shapes share, such as position and number of sides. It also includes abstract methods such as getArea(), getPerimeter() and display(), which are completed in the subclasses.

The Rectangle, Square, Circle and Triangle classes all inherit from Shape. Each one has its own extra attributes. For example, Rectangle has width and length, Square has side, Circle has radius, and Triangle has three vertices. Each subclass overrides the methods from Shape so that the correct area, perimeter, scaling and display behaviour is used.

The ShapeList class stores the shapes in an ArrayList <Shape>. This allows different types of shapes to be kept in the same list. The ShapeManagement class contains the menu that the user interacts with. It allows the user to add shapes, remove shapes, display shape information, calculate area and perimeter, translate all shapes and scale all shapes.

The UML class diagram was created using PlantUML and shows the relationship between the classes:



PNG | SVG | PDF | TXT | Share

Implementation

The implementation follows the UML design. I started with the Coordinates class because it is needed by the other classes. After that, I implemented Shape as an abstract superclass with shared attributes and methods.

The four shape classes extend Shape and each class carries out its own calculations.

- Rectangle calculates area using width multiplied by length and perimeter using twice the width plus twice the length.
- Square calculates area using side multiplied by side and perimeter using four times the side.
- Circle uses Math.PI to calculate area and perimeter.
- Triangle uses the distances between its three vertices to calculate perimeter and Heron's Formula to calculate area (https://mathworld.wolfram.com/HeronsFormula.html)

The ShapeList class manages the list of shapes and checks positions before getting or removing a shape. This is important because it stops the program from crashing if the user enters a position that does not exist. The ShapeManagement class uses a loop and a switch statement to show the menu and run the option chosen by the user.

Problems Encountered and Solutions

One problem I had to consider was how to deal with invalid positions. For example, if the user tries to remove a shape at position 30 when there are not that many shapes in the list, the program should not crash. I solved this by checking the position before using it in methods such as getShape() and removeShape().

Another issue was the Triangle class. The other shapes only need one position, but a triangle has three vertices. This meant that translating a triangle had to move all three vertices, not just the inherited position. I solved this by overriding the translate() method in the Triangle class.

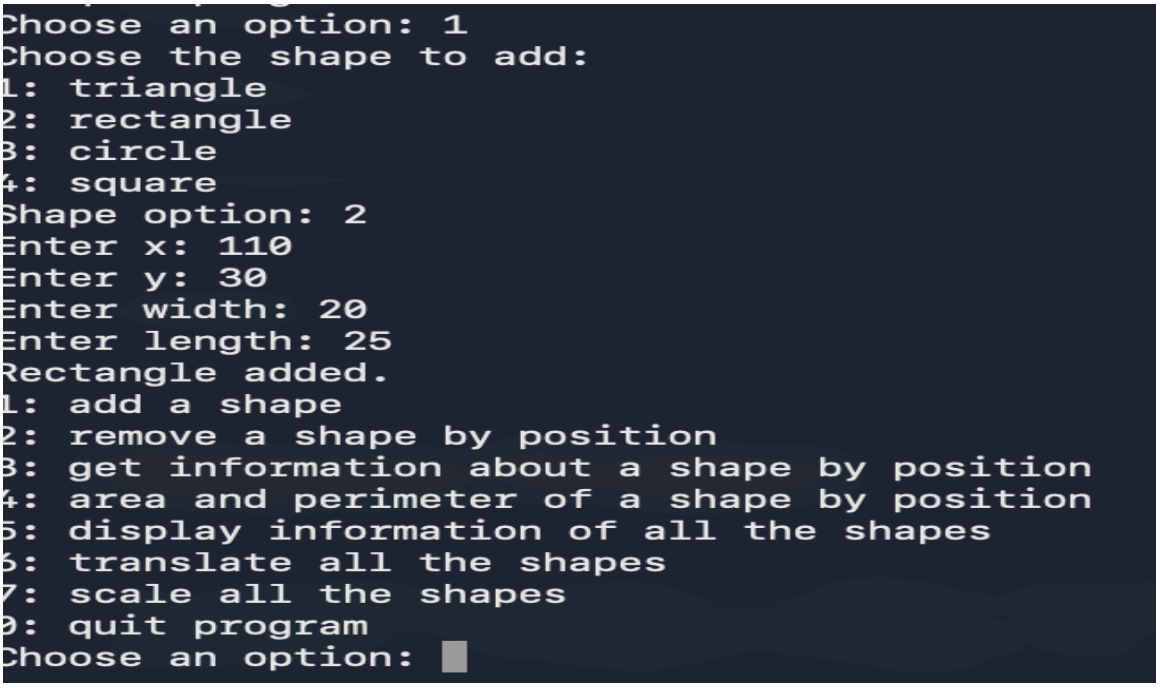
Test action	Expected result	Actual result
Add triangle with vertices (60,60), (30,80), (80,80)	The triangle should be added to the list. Its three vertices should be displayed correctly. The perimeter should be approximately 114.34 and the area should be 500.0.	The triangle was added successfully and displayed with the correct vertices. The calculated area and perimeter matched the expected result.
Add rectangle at position (110,30)with width 20 and length 25	The rectangle should be added to the list. The area should be 500.0 and the perimeter should be 90.0.	The rectangle was added successfully. The program displayed area 500.0 and perimeter 90.0, which matched the expected result.
Add circle at position (90,110)with radius 35	The circle should be added to the list. The area should be approximately 3848.45 and the perimeter should be approximately 219.91.	The circle was added successfully and the displayed area and perimeter matched the expected result.
Add square at position (100,50)with side 30	The square should be added to the list. The area should be 900.0 and the perimeter should be 120.0.	The square was added successfully. The program displayed area 900.0 and perimeter 120.0, which matched the expected result.
Add three additional shapes of my choice	Three more shapes should be added to the list, increasing the total number of shapes.	Three additional shapes were added successfully, and the list displayed the increased number of shapes.
Show the area and perimeter of the second shape in the list	The program should display the area and perimeter of the second shape only.	The program displayed the area and perimeter of the second shape correctly.

Display information for all shapes	The program should display every shape in the list, including its coordinates, dimensions, area and perimeter.	The program displayed all shapes in the list clearly with their details.
Remove the third shape from the list	The third shape should be removed from the list and the program should continue running without an error.	The third shape was removed successfully and the program continued running without crashing.
Translate all shapes by x = 20 and y = 25	All shapes should move by 20 in the x direction and 25 in the y direction. For the triangle, all three vertices should move.	All shapes were translated correctly. The triangle vertices and other shape positions were updated as expected.
Display information for all shapes after translation	The updated coordinates should be shown for all shapes.	The program displayed the new coordinates for all shapes after translation.
Increase all shapes by a scale factor of 5	All coordinates and dimensions should be multiplied by 5.	The shapes were scaled successfully and the displayed values changed as expected.
Display all shapes after scaling	The program should show the updated scaled positions and dimensions for all shapes.	The program displayed all scaled shapes correctly.
Remove a shape at invalid position 30	The program should show a clear invalid position message and should not crash.	The program showed an invalid position message and continued running.
Display the area of a shape at invalid position 110	The program should show a clear invalid position message and should not crash.	The program handled the invalid area request safely and continued running.

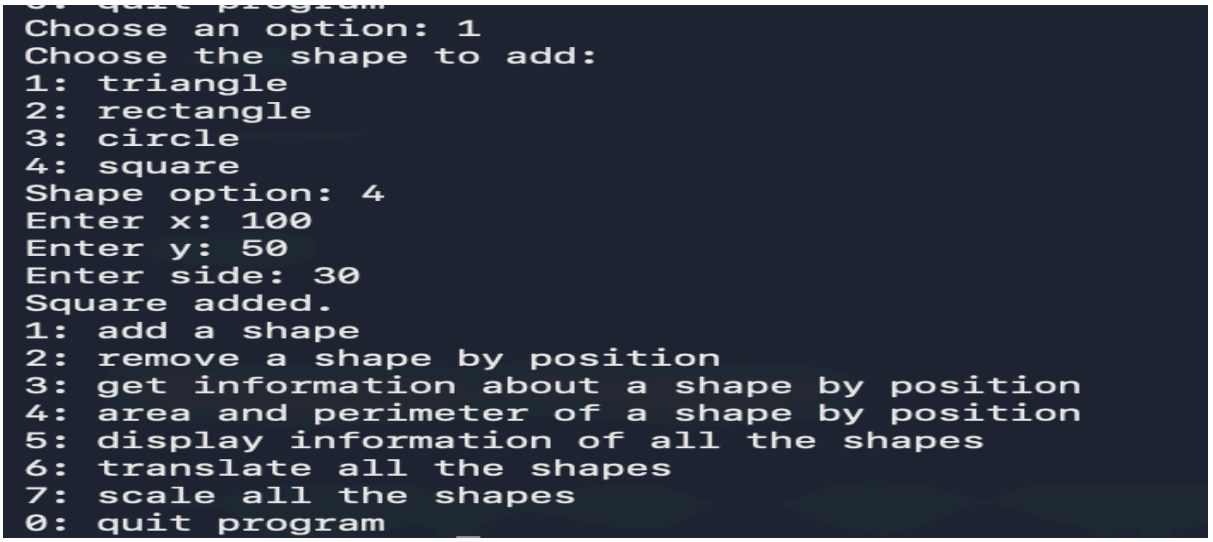
TEST FOR THE PROGRAM

Implementing the project for testing

Input for Rectangle



Input for Square



Input for circle

```
Choose the shape to add:
1: triangle
2: rectangle
3: circle
4: square
Shape option: 3
Enter x: 90
Enter y: 110
Enter radius: 35
Circle added.
1: add a shape
2: remove a shape by position
3: get information about a shape by position
4: area and perimeter of a shape by position
5: display information of all the shapes
6: translate all the shapes
7: scale all the shapes
0: quit program
```

Scale All shapes by 5

```
Choose an option: 7
Enter scale factor: 5
Enter 1 to multiply or 0 to divide: 1
All shapes scaled.
```

Input for triangle

```
Choose the shape to add:
1: triangle
2: rectangle
3: circle
4: square
Shape option: 1
Enter vertex 1:
Enter x: 60
Enter y: 60
Enter vertex 2:
Enter x: 30
Enter y: 80
Enter vertex 3:
Enter x: 80
Enter y: 80
Triangle added.
1: add a shape
2: remove a shape by position
3: get information about a shape by position
4: area and perimeter of a shape by position
5: display information of all the shapes
6: translate all the shapes
7: scale all the shapes
0: quit program
```

Displays the information of all the shapes

```
Choose an option: 5
Position 1: Rectangle: position (X = 110, Y = 30), width = 20, length = 25, area = 500.0, perimeter = 90.0
Position 2: Square: position (X = 100, Y = 50), side = 30, area = 900.0, perimeter = 120.0
Position 3: Circle: centre (X = 90, Y = 110), radius = 35, area = 3848.4510006474966, perimeter = 219.9114857512855
Position 4: Triangle: vertex 1 (X = 60, Y = 60), vertex 2 (X = 30, Y = 80), vertex 3 (X = 80, Y = 80), area = 500.0, pe
imeter = 114.3397840021018
```

```
Enter the position: 2
Area: 900.0
Perimeter: 120.0
```

The results matched the expected behaviour, including the error-handling tests for invalid positions/

Reflection on OOP Principles

This project helped me understand how object-oriented programming can make a program more organised. Encapsulation was used by keeping attributes private and using methods to access or change them. This makes the classes easier to control and reduces the chance of values being changed incorrectly.

Abstraction was used through the Shape class. Since Shape is only a general idea, it does not calculate its own area or perimeter. Instead, the subclasses provide the specific calculations.

Inheritance was useful because Rectangle, Square, Circle and Triangle could all reuse the common features from Shape. This avoided repeating the same code in every class.

Polymorphism was used in ShapeList because the list stores objects as Shape, even though the actual objects may be rectangles, squares, circles or triangles. This makes the program more flexible because different shapes can be handled in the same way.

References

Liang, Y.D. (2021) *Introduction to Java Programming and Data Structures*. 12th edn. Pearson.

Oracle (2026) *Java Platform, Standard Edition Documentation*. Available at: https://docs.oracle.com/en/java/javase/22/docs/api/index.html